Delay and Ring Buffer

Intro to Audio Tech 2
Jocelyn Kavanagh
Tuesday, April 14

Georgia Tech

Lesson Outline

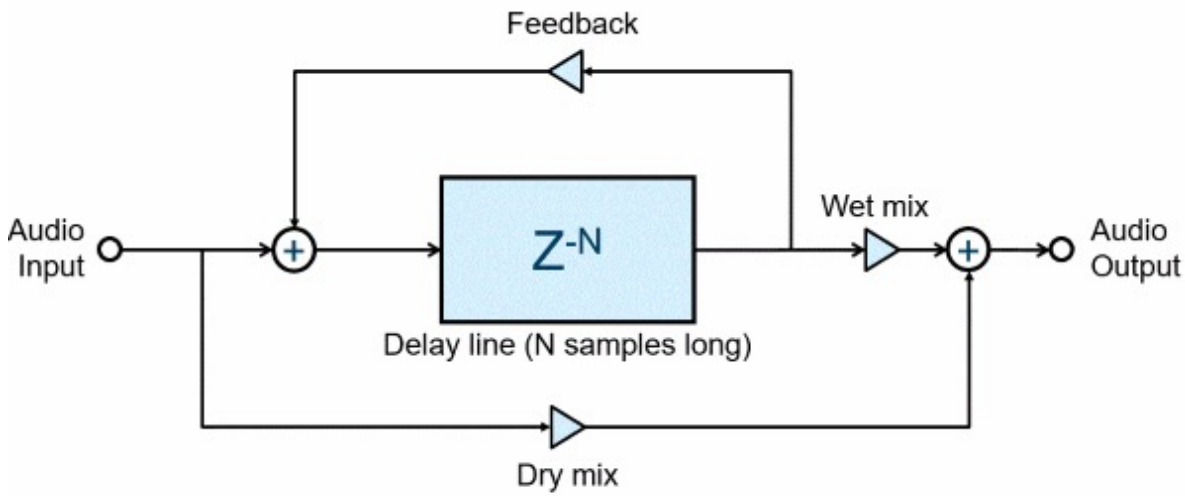
- Implementing Delay
- Using a ring buffer

Georgia Tech

Delay

Delay

- Remember when implementing delay, we store or copy samples and add to the outgoing signal



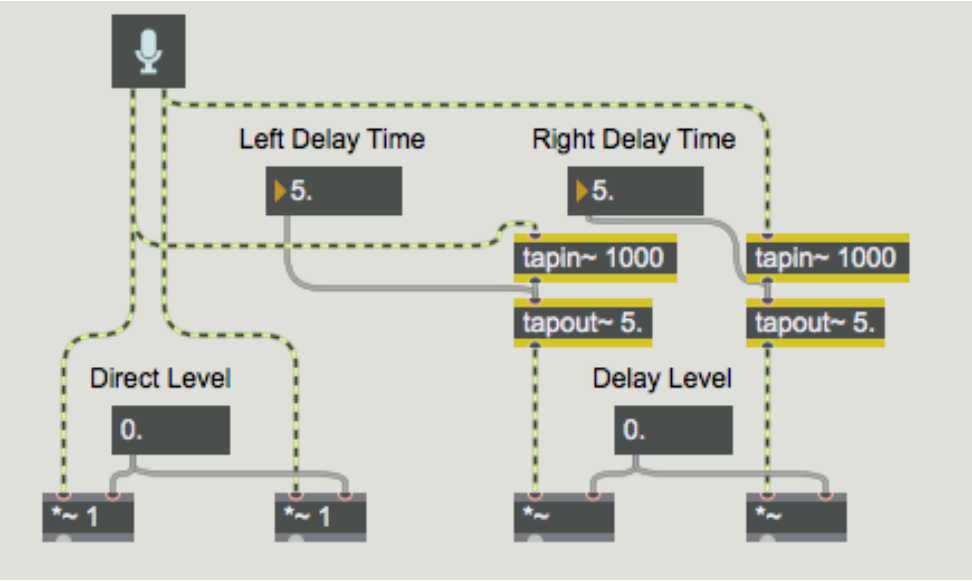
Delay

- In offline Python, this looked like copying the input array into a new array that has empty samples in the beginning

```
copy = data.copy()
#time in ms to delay by
offset = 700
#create number of samples equal to 700ms under the given sampling rate
pad = np.zeros(int(fs/1000*offset))
orig = np.concatenate([data,pad])
delay = np.concatenate([pad,copy])
new_s = orig+delay
```

Delay

- In real time Max, this looked like defining a max delay length with tapin~ and accessing the delay buffer at a given delay length with tapout~



Simple Delay

- Our approach in C++ will be similar to that in Max
  - Really, it's how Max is implemented under the hood
- In our block-based processing set-up, we only have access to the current I/O buffer by default
  - Audio arrives in blocks, not sample-by-sample
  - We typically process only the current buffer
- We need a way to store past samples, so we will define past samples as a member variable of our AudioProcessor that we will continuously update
- Let's start with a single sample delay!

Longer Delay Line

- What if we wanted a longer delay?
- Instead of storing an individual sample, we could store a delay buffer and update continuously
- Our delay can never be longer than the length of our delay buffer (like in Max with tap in and tap out)
- We'll use a FIFO structure
- Let's try a multi-sample delay

Newest samples			Oldest samples
0	0	0	0
1	0	0	0
2	1	0	0
3	2	1	0
4	3	2	1
5	4	3	2

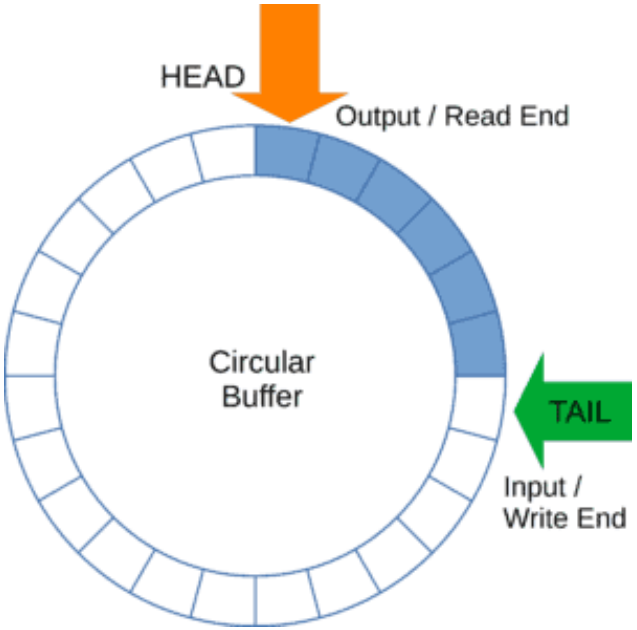
Longer Delay Line

- If we try saving the past samples into a buffer, we need to shift the contents and update that buffer each time we read into the delay line
- This may seem like a good idea and will work in offline applications or very short delay lengths, but it does not work for long delay lines in real-time settings
- By copying over and updating the buffer each time we update a new sample, we increase our time complexity by O(N * M) where N is the length of the buffer size and M is the length of the delay line
- This is too long!
  - We get block boundary issues, scheduling jitter, unpredictable timing, audible artifacts

Ring Buffer

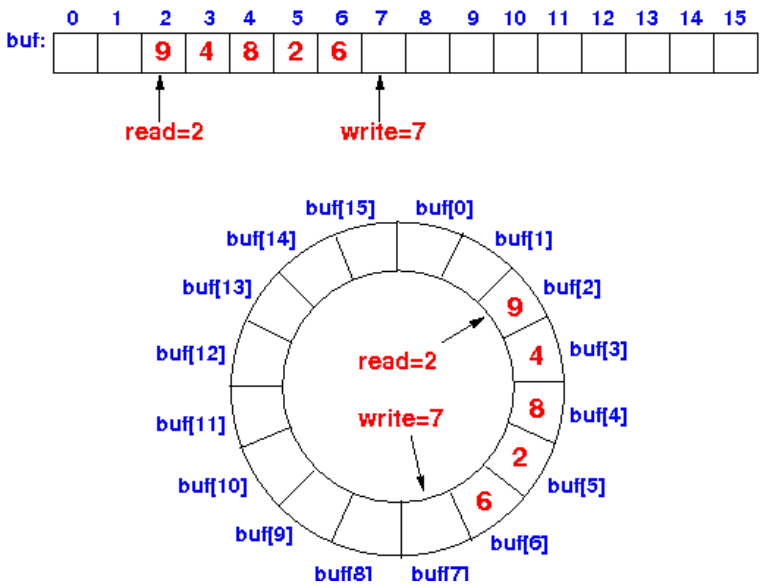
Ring Buffer

- Rather than shifting all of our buffer contents each cycle, we can use a “new” data structure that helps us keep more fluid storage
- A ring buffer (also called circular buffer) takes a fixed buffer and treats it as a circle
- Instead of tracking the beginning or end of the buffer, we’ll track read and write positions



Ring Buffer

- We'll store/push new samples at the write index and read/"pop" old samples with the read index
- The read index will always be "behind" the write index
- In our real-time audio context, our read and write index will move at the same rate (for now)
- Our read and write indexes will wrap around the buffer length back to 0

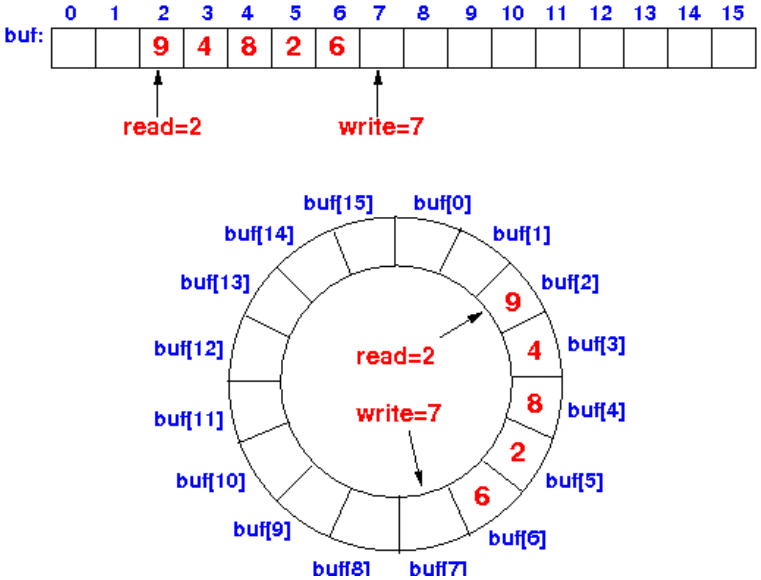


Ring Buffer

- Our read index relies on
  - Our delay length in samples
  - Our delay buffer size
  - Our write index
- We will calculate the read index with

int readIndex = ((writeIndex – delayInSamples) + delayBufferSize) % delayBufferSize;

- This wraps the read index around the buffer but keeps it away from our write index by our delay sample length



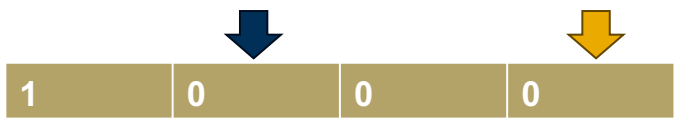
Delay with Ring Buffer

- Each iteration across our process block samples we will calculate the read index then increment the write index then update the output

Write starts at ind0
Read = ((writeIndex – delayInSamples) + delayBufferSize) % delayBufferSize
Read = (0 - 2 + 4) % 4 = ind2
Store at write index

Output 0

Write now at ind1
Read = ((1 – 2) + 4) % 4 = ind3
Store at write index



Output 0

Write now at ind2
Read = ((2 – 2) + 4) % 4 = ind0
Store at write index

Output 1

Delay with Ring Buffer

- Each iteration across our process block samples we will calculate the read index then increment the write index then update the output

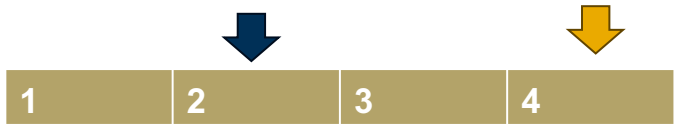
Write now at ind3
Read = ((3 – 2) + 4) % 4 = ind1
Store at write index

Output 2

Write now at ind0
Read = ((0 – 2) + 4) % 4 = ind2
Store at write index

Output 3

Write now at ind1
Read = ((1 – 2) + 4) % 4 = ind3
Store at write index



Output 4

Georgia Tech